

管理者アカウントによる打刻申請の承認・差戻し機能

まずは、この「承認・差戻し機能」をどのようなトラフィック（データの流れ）と画面構造で実現するか、概要とバックエンドの土台から考察していきましょう。

管理者承認機能のトラフィック（仕組み）の考察

一般ユーザーが申請したデータを、管理者が処理する際のデータの動きは以下のようになります。

- **管理者が一覧を見る:**

管理者が「申請一覧画面」を開くと、システム全体の `EntryExitCalendar` テーブルから、`isTimechangeDemand = 1`（申請中）になっているレコードだけが抽出されて表示されます。

- **管理者が「承認」ボタンを押した場合:**

- `timechangeStatus` を `2`（承認済）に更新します。
- **【重要】** ユーザーが希望した修正時刻（`tmpEntryTime` / `tmpExitTime`）の値を、本番の「確定時刻（`entryTime` / `exitTime`）」の枠にパチッと上書きコピーします。
- `isTimechangeDemand` を `0`（申請終了）に戻します。

- 管理者が「差戻し」ボタンを押した場合:

- `timechangeStatus` を `1`（差戻し）に更新します。
- なぜダメだったのかの理由（「打刻原簿と乖離しています」など）を、`adminComment` カラムに書き込みます。
- 確定時刻は書き換えずにそのまま残します。

このように処理することで、一般ユーザーが自分のカレンダー画面を見たときに、「あ、承認されて確定時間が変わった！」「あ、差戻されて管理者からのコメントが届いている！」という双方向のキャッチボールが成立します。

バックエンド（Java）：リポジトリへの「窓口」追加

まずは、管理者が「今、誰が申請中なのか」をデータベースから一斉に掘り起こすための新しいお願い（メソッド）を、カレンダー用リポジトリに追加しましょう。

EntryExitCalendarRepository.java への追加記述

```
// 管理者用：日本全国の全ユーザーの中から、「時刻修正申請中 (isTimechangeDemand = 1)」の行だけを全て取ってくる命令
List<EntryExitCalendar> findByIsTimechangeDemandOrderByRecordDateAsc(int isTimechangeDemand);
```

バックエンド (Java)：Service層へのロジック肉付け

続いて、新しく `AdminAttendanceService.java` を `service` パッケージ配下に作成（または既存の管理者用サービスに追記）します。

ここには「承認する処理」と「差戻す処理」の2つの具体的な業務ロジックを記述します。

```
package com.appspace.backend.service;

import com.appspace.backend.entity.EntryExitCalendar;
import com.appspace.backend.repository.EntryExitCalendarRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
```

```
import java.util.List;
import java.util.Optional;

@Service
@Transactional
public class AdminAttendanceService {

    @Autowired
    private EntryExitCalendarRepository calendarRepository;

    /**
     * 管理者ロジック1: 現在申請が届いているレコードを全件取得する
     */
    public List<EntryExitCalendar> getPendingTimechangeRequests() {
        return calendarRepository.findByIsTimechangeDemandOrderByRecordDateAsc(1); // 「1: 申請中」のものを利口に全件取得
    }

    /**
     * 管理者ロジック2: 申請を【承認】する
     * @param recordId 対象のレコードID (RECxxxx)
     */
    public void approveTimechange(String recordId) {
        EntryExitCalendar record = calendarRepository.findById(recordId)
            .orElseThrow(() -> new RuntimeException("指定された申請レコードが見つかりません。"));

        // データの引越し: 仮保存されていた希望時刻を、本番の確定時刻枠にコピー！
        record.setEntryTime(record.getTmpEntryTime());
        record.setExitTime(record.getTmpExitTime());

        // ステータスを「2: 承認済み」にする
        record.setTimechangeStatus(2);
        // 処理が完了したので、申請中フラグを「0 (通常状態)」に下げる
        record.setIsTimechangeDemand(0);

        calendarRepository.save(record); // データベースを確定更新！
        System.out.println("=== [Admin Logic] 申請の承認および時刻の確定上書きが完了しました ===");
    }
}
```

```

/**
 * 管理者ロジック3: 申請を【差戻し（却下）】する
 * @param recordId 対象のレコードID
 * @param adminComment 管理者からの言い分・理由
 */
public void rejectTimechange(String recordId, String adminComment) {
    EntryExitCalendar record = calendarRepository.findById(recordId)
        .orElseThrow(() -> new RuntimeException("指定された申請レコードが見つかりません。"));

    // ステータスを「1：差戻し」にする
    record.setTimechangeStatus(1);
    // ユーザーが再修正できるように、申請中フラグは「1（申請状態）」のままキープ、または仕様に合わせて調整
    // 今回はユーザー画面側の「再申請」ボタン活性化のために1のまま、あるいは状態を維持します
    record.setAdminComment(adminComment); // 差戻し理由を格納

    calendarRepository.save(record);
    System.out.println("=== [Admin Logic] 申請の差戻し処理が完了しました ===");
}
}

```

バックエンド（Java）：Controller層（管理者用窓口）の新設

最後に、管理者用画面からの通信を受け付ける `AdminAttendanceController.java` を `controller` パッケージ配下に新設します。

```
package com.appspace.backend.controller;

import com.appspace.backend.entity.EntryExitCalendar;
import com.appspace.backend.service.AdminAttendanceService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/admin/attendance")
@CrossOrigin(origins = "https://*.app.github.dev")
public class AdminAttendanceController {

    @Autowired
    private AdminAttendanceService adminService;

    /**
     * 管理者窓口1: 未承認の編集申請一覧を一括取得する
     * GET http://localhost:8080/api/admin/attendance/requests
     */
    @GetMapping("/requests")
    public ResponseEntity<List<EntryExitCalendar>> getPendingRequests() {
        return ResponseEntity.ok(adminService.getPendingTimechangeRequests());
    }
}
```

```

/**
 * 管理者窓口2: 申請を承認する
 * POST http://localhost:8080/api/admin/attendance/approve
 */
@PostMapping("/approve")
public ResponseEntity<String> approveRequest(@RequestBody Map<String, String> payload) {
    String recordId = payload.get("recordId");
    try {
        adminService.approveTimechange(recordId);
        return ResponseEntity.ok("申請を承認しました。打刻時間が更新されました。");
    } catch (Exception e) {
        return ResponseEntity.internalServerError().body("承認処理に失敗しました: " + e.getMessage());
    }
}

/**
 * 管理者窓口3: 申請を差し戻す
 * POST http://localhost:8080/api/admin/attendance/reject
 */
@PostMapping("/reject")
public ResponseEntity<String> rejectRequest(@RequestBody Map<String, String> payload) {
    String recordId = payload.get("recordId");
    String comment = payload.get("adminComment");

    if (comment == null || comment.trim().isEmpty()) {
        return ResponseEntity.badRequest().body("差し戻し理由（コメント）を入力してください。");
    }

    try {
        adminService.rejectTimechange(recordId, comment);
        return ResponseEntity.ok("申請を差し戻しました。");
    } catch (Exception e) {
        return ResponseEntity.internalServerError().body("差し戻し処理に失敗しました: " + e.getMessage());
    }
}
}

```


SecurityConfig.java へのURL追加確認

この管理者専用パスへのアクセスを許可するため、SecurityConfig.java に以下のパスを追加して保護（あるいは許可リストへの追加）を行ってください。

```
"/api/admin/attendance/**", // ★管理者用の打刻承認パスを追加許可
```

これで「管理者が申請を覗き見し、ボタン一つで承認（時間を書き換え）または差戻し（コメント付与）する」ためのバックエンド側の強固な防壁とロジックがすべて揃いました！

フロントエンド側の実装

それでは、この裏方窓口（API）と通信して、管理者がブラウザ上でポチポチと承認・差戻しを操作できる管理者用フロントエンド画面（AdminAttendanceApprovalView.vue）の実装に進みましょう！

一般ユーザー側の画面と同様に、初学者の方でも処理のトラフィック（データの流れ）を完全に追いかけるよう、詳細なコメントを網羅したコードをご提示します。

フロントエンド：AdminAttendanceApprovalView.vue の実装

src/views/（または管理者用コンポーネントを取りまとめているディレクトリ）に
AdminAttendanceApprovalView.vue を新規作成し、以下のコードを記述してください。

```
<template>
  <div class="admin-approval-container">
    <h2>勤怠・打刻修正承認パネル（管理者専用）</h2>
    <p class="subtitle">全国の従業員から提出された「打刻内容の修正申請」の一覧確認と、承認・差戻し処理が行えます。</p>

    <div v-if="pendingRequests.length === 0" class="no-data-alert">
      現在、未処理の修正申請はありません。
    </div>

    <div v-else class="requests-grid">
      <div v-for="req in pendingRequests" :key="req.recordId" class="request-card">
        <div class="card-header">
          <span class="user-id">社員ID: {{ req.registeredAccountId }}</span>
          <span class="target-date">対象日: {{ formatDate(req.recordDate) }}</span>
        </div>

        <div class="card-body">
          <div class="time-comparison-box">
            <div class="time-row origin-time">
              <span class="time-label">現在の確定打刻</span>
              <span class="time-values">Entry: <b>{{ req.entryTime }}</b> / Exit: <b>{{ req.exitTime }}</b></span>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
```

```

    </div>
    <div class="arrow-down">修正希望（この内容へ差し替え） </div>
    <div class="time-row target-time">
      <span class="time-label">修正後の希望</span>
      <span class="time-values">Entry: <b class="highlight">{{ req.tmpEntryTime }}</b> / Exit: <b class="highlight">{{ req.tmpExitTime }}</b></span>
    </div>
  </div>

  <div class="reason-section">
    <strong>従業員からの申請理由・備考:</strong>
    <p class="reason-text">{{ req.reason }}</p>
  </div>

  <div v-if="activeRejectId === req.recordId" class="reject-comment-area">
    <label>差戻し理由を入力してください <span class="required">※必須</span></label>
    <textarea
      v-model="adminComment"
      placeholder="例： 打刻原簿のログと時間が大幅に乖離しています。再度確認して申請し直してください。"
      rows="2"
    ></textarea>
  </div>
</div>

<div class="card-actions">
  <template v-if="activeRejectId !== req.recordId">
    <button @click="handleApprove(req.recordId)" class="btn-approve">承認する</button>
    <button @click="showRejectInput(req.recordId)" class="btn-trigger-reject">差戻す</button>
  </template>

  <template v-else>
    <button @click="handleReject(req.recordId)" class="btn-reject-confirm">この理由で差戻しを確定</button>
    <button @click="cancelReject" class="btn-cancel">キャンセル</button>
  </template>
</div>
</div>
</div>
</template>

```

```

<script setup>
import { ref, onMounted } from 'vue';
import apiClient from '../api';

// 状態管理用変数
const pendingRequests = ref([]); // 届いている未承認申請のリスト
const activeRejectId = ref(null); // 現在どこかの行の「差戻し理由」を入力中かをIDで保持
const adminComment = ref(''); // 管理者が入力する差戻しコメント

// 1. サーバーから未承認の申請一覧をシュッと引っ張ってくる関数
const fetchPendingRequests = async () => {
  try {
    const response = await apiClient.get('/admin/attendance/requests');
    pendingRequests.value = response.data;
  } catch (error) {
    console.error('申請一覧の取得に失敗しました:', error);
    alert('申請データの取得に失敗しました。');
  }
};

// 2. 【承認ボタン】が押されたときの処理
const handleApprove = async (recordId) => {
  if (!confirm('この修正申請を承認してもよろしいですか？\n(承認すると、本番のカレンダー確定時刻が書き換わります)')) {
    return;
  }

  try {
    // バックエンドの POST /api/admin/attendance/approve ^ recordId を送信
    const response = await apiClient.post('/admin/attendance/approve', { recordId });
    alert(response.data); // 「申請を承認しました」メッセージを表示
    fetchPendingRequests(); // 処理が終わったのでリストを最新に更新
  } catch (error) {
    console.error('承認エラー:', error);
    alert('承認処理に失敗しました。');
  }
};

```

```
// 差戻し入力欄の表示切り替え
const showRejectInput = (recordId) => {
  activeRejectId.value = recordId;
  adminComment.value = ''; // コメント欄をクリア
};

const cancelReject = () => {
  activeRejectId.value = null;
  adminComment.value = '';
};

// 3. 【差戻し確定ボタン】が押されたときの処理
const handleReject = async (recordId) => {
  if (!adminComment.value.trim()) {
    alert('従業員へ伝える「差戻し理由」を入力してください。');
    return;
  }

  try {
    // バックエンドの POST /api/admin/attendance/reject へ送信
    const response = await apiClient.post('/admin/attendance/reject', {
      recordId: recordId,
      adminComment: adminComment.value
    });
    alert(response.data); // 「申請を差戻しました」を表示
    cancelReject(); // 入力状態を閉じる
    fetchPendingRequests(); // リストを最新に更新
  } catch (error) {
    console.error('差戻しエラー:', error);
    alert('差戻し処理に失敗しました。');
  }
};

// 日付を「〇月〇日」に見やすく整形する関数
const formatDate = (dateStr) => {
  if (!dateStr) return '';
  const [y, m, d] = dateStr.split('-');
  return `${Number(m)}月${Number(d)}日`;
};

// 画面展開時に自動で申請一覧をロード
onMounted(() => {
  fetchPendingRequests();
});
</script>
```

```
<style scoped>
.admin-approval-container {
  max-width: 850px;
  margin: 40px auto;
  padding: 20px;
  font-family: sans-serif;
  color: #333;
}
.subtitle { color: #666; margin-bottom: 30px; }

.no-data-alert {
  background-color: #e8f5e9;
  color: #2e7d32;
  padding: 20px;
  border-radius: 6px;
  text-align: center;
  font-weight: bold;
  font-size: 1.1rem;
  border: 1px solid #c8e6c9;
}

/* 申請カードのレイアウト */
.requests-grid { display: flex; flex-direction: column; gap: 20px; }
.request-card {
  background: white;
  border-radius: 8px;
  border: 1px solid #cfd8dc;
  box-shadow: 0 4px 10px rgba(0,0,0,0.05);
  overflow: hidden;
  text-align: left;
}
```

```
.card-header {
  background-color: #37474f;
  color: white;
  padding: 12px 20px;
  display: flex;
  justify-content: space-between;
  font-weight: bold;
  font-size: 0.95rem;
}

.card-body { padding: 20px; }

/* 時刻比較ボックスの装飾 */
.time-comparison-box { background: #f8f9fa; border: 1px dashed #b0bec5; border-radius: 6px; padding: 15px; margin-bottom: 15px; }
.time-row { display: flex; justify-content: space-between; font-size: 0.95rem; padding: 4px 0; }
.origin-time { color: #78909c; }
.target-time { font-size: 1.05rem; font-weight: bold; color: #2c3e50; }
.arrow-down { text-align: center; font-size: 0.85rem; color: #007bff; margin: 4px 0; font-weight: bold; }
.highlight { color: #d32f2f; background: #ffebee; padding: 2px 6px; border-radius: 4px; font-family: monospace; }
.time-values b { font-family: monospace; font-size: 1.05rem; }

/* 理由表示エリア */
.reason-section { background: #fffde7; border-left: 4px solid #fbc02d; padding: 10px 15px; border-radius: 4px; margin-bottom: 15px; }
.reason-section strong { font-size: 0.9rem; color: #f57f17; }
.reason-text { margin: 6px 0 0 0; font-size: 0.95rem; line-height: 1.4; color: #424242; }

/* 差戻しエリア */
.reject-comment-area { margin-top: 15px; background: #fff3e0; padding: 12px; border-radius: 4px; border: 1px solid #ffe0b2; }
.reject-comment-area label { display: block; font-weight: bold; font-size: 0.85rem; color: #e65100; margin-bottom: 6px; }
.reject-comment-area textarea {
  width: 100%;
  padding: 8px;
  border: 1px solid #ccc;
  border-radius: 4px;
  box-sizing: border-box;
  font-size: 0.95rem;
}
.required { color: #d32f2f; }
```

```
/* 各種ボタンのスタイリング */
.card-actions {
  background: #f5f7f8;
  padding: 12px 20px;
  display: flex;
  gap: 12px;
  justify-content: flex-end;
  border-top: 1px solid #eceff1;
}

.btn-approve {
  background-color: #4caf50;
  color: white;
  border: none;
  padding: 8px 20px;
  border-radius: 4px;
  font-weight: bold;
  cursor: pointer;
  transition: background 0.2s;
}
.btn-approve:hover { background-color: #388e3c; }

.btn-trigger-reject {
  background-color: white;
  color: #d32f2f;
  border: 1px solid #d32f2f;
  padding: 8px 20px;
  border-radius: 4px;
  font-weight: bold;
  cursor: pointer;
  transition: all 0.2s;
}
.btn-trigger-reject:hover { background-color: #ffebee; }
```



```
.btn-reject-confirm {  
  background-color: #e65100;  
  color: white;  
  border: none;  
  padding: 8px 20px;  
  border-radius: 4px;  
  font-weight: bold;  
  cursor: pointer;  
}  
.btn-reject-confirm:hover { background-color: #c62828; }  
  
.btn-cancel {  
  background-color: #cfd8dc;  
  color: #37474f;  
  border: none;  
  padding: 8px 16px;  
  border-radius: 4px;  
  cursor: pointer;  
}  
.btn-cancel:hover { background-color: #b0bec5; }  
</style>
```

router/index.js (Vue Router) への追加

この管理者画面にURLでアクセスできるよう、ルーティング（URLパス）の登録を行きましょう。

router/index.js の中に、新しく以下を追加してください。

```
{
  path: '/admin/approval',
  name: 'AdminAttendanceApproval',
  component: () => import('../views/AdminAttendanceApprovalView.vue')
}
```

双方向キャッチボールの感動的なテスト手順

フロント・バックエンドが組み合わさった状態を、以下の手順でテストしてみましょう！

- 【従業員側】 申請を出す:

一般ユーザーでログインし、打刻内容編集申請画面（AttendanceEditView.vue）から「5月27日」などの行を「09:30:00 ～ 18:45:00 / 理由： 電車の遅延のため」で編集申請します。

- 画面上に「申請中 (未承認)」のオレンジバッジが点灯します。

- **【管理者側】 承認パネルを開く:**

ブラウザのURLを `/admin/approval` に手動で切り替えて、新しく作った管理者画面を開きます。

- 画面に、先ほど従業員が出した「電車の遅延のため」というカードがリアルタイムで1件ポコッと出現します！

- **【管理者側】 承認を実行する:**

そのカードの「承認する」ボタンを押し、アラートでOKをクリックします。

- カードが画面から消え、「未処理の申請はありません」の表示に戻ります。

- **【従業員側】 結果を確認する:**

再び従業員側のカレンダー画面に戻ってみてください。

- オレンジ色だったバッジが、見事に「承認済み」の緑色バッジに変化します。
- そして、その日の確定Entry/Exit時刻が、申請通り `09:30:00` と `18:45:00` にガチッと上書き更新されているのが確認できます！